

01 / EINORDNUNG

Vom Rechnen zum Addierer

Ziel: Du baust einen 4-Bit-Addierer und erklärst, warum mehr Bits bei einer einfachen Übertragskette mehr Wartezeit bedeuten. Python vergleicht zwei bewusst vereinfachte Zeitmodelle.

Literatur: Harris & Harris, *Digital Design and Computer Architecture, RISC-V Edition*, Kapitel 5.2.1, Buchseiten 237–244, PDF-Seiten 260–267. Kernlektüre: Buchseiten 238–243. Eigene Erklärungen und Abbildungen in diesem Blatt.

Verbindung zu Tag 4 und 10: Die Binäraddition kennst du bereits. Heute baust du die Hardware dafür. Aus Tag 10 weißt du: Ein Gatter liefert sein neues Ergebnis erst nach einer Verzögerung.

Notation: A_i, B_i sind die Eingangsbits an Stelle i . Bit 0 ist das niederwertigste Bit. Wir nennen den Übertrag in Stelle i hier c_i : $c_0 = C_{in}$ und bei vier Bits $c_4 = C_{out}$. Das Buch verwendet stellenweise den Index des erzeugenden Bits; unsere Beschriftung bleibt durchgehend gleich.

Stelle	3	2	1	0
Stellenwert	8	4	2	1
$A = 7$	0	1	1	1
$B = 1$	0	0	0	1
Summe	1	0	0	0

In Stelle 0 gilt $1 + 1 = 10_2$: Summe 0, Übertrag 1. Dieser Übertrag wird in Stelle 1 mitgerechnet. Dort entsteht erneut ein Übertrag; ebenso in Stelle 2. In Stelle 3 ergibt $0 + 0 + 1$ schließlich die Summe 1.

Eine ganze Addition prüfen:

$$A + B + C_{in} = S + 2^N C_{out}$$

Für unsigned 4-Bit-Eingänge ist S zwischen 0 und 15. Beispiel: $15 + 1 = 0 + 16 \cdot 1$. Der fünfte Ergebnisbit steckt in C_{out} .

Carry ist nicht signed Overflow. Bei vier Bits ist $7 + 1$ signed nicht darstellbar, obwohl kein Carry-out entsteht. Bei unsigned $15 + 1$ entsteht dagegen Carry-out. Heute prüfen wir den Addierer zuerst unsigned; seine Bits können anschließend auch als Zweierkomplement interpretiert werden.

Heute selbst umsetzen: Volladdierer, vier verbundene Stufen, Tests und zwei Python-Funktionen. Einen Prefix-Addierer musst du heute nicht verdrahten; HDL und Transistorschaltungen sind ebenfalls kein neuer Pflichtteil.

02 / EIN BIT

Halbaddierer und Volladdierer

Ein Halbaddierer addiert nur A und B :

$$S = A \oplus B, \quad C_{out} = A \wedge B.$$

$\oplus$ bedeutet XOR. Bei $1 + 1$ ist $S = 0$ und $C_{out} = 1$. Ein Halbaddierer kann keinen Übertrag aus einer vorherigen Stelle annehmen.

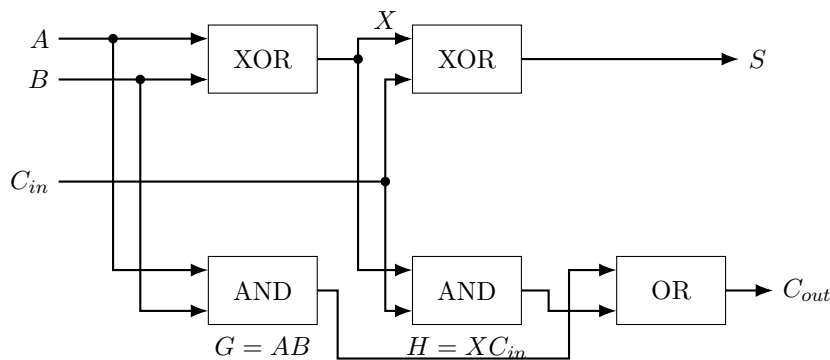
Ein **Volladdierer** addiert drei einzelne Bits: A, B, C_{in} . Die Summe kann 0, 1, 2 oder 3 betragen. Er gibt sie als zwei Bits aus: $2C_{out} + S$.

A	B	C_{in}	S	C_{out}	Dezimalsumme
0	0	0	0	0	0
0	0	1	1	0	1
0	1	0	1	0	1
0	1	1	0	1	2
1	0	0	1	0	1
1	0	1	0	1	2
1	1	0	0	1	2
1	1	1	1	1	3

Für den Bau verwenden wir diese fünf Gatter:

$$X = A \oplus B, \quad S = X \oplus C_{in}, \quad G = AB, \quad H = XC_{in}, \quad C_{out} = G + H.$$

Hier bedeutet das Pluszeichen in einer Logikgleichung OR, nicht eine arithmetische Addition.



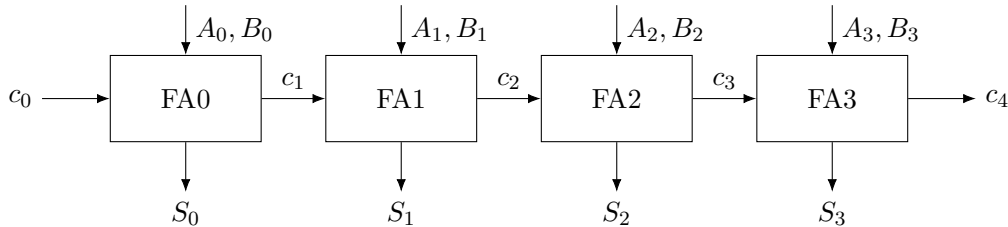
Das ist ein zusammenhängender Volladdierer: Es gibt genau drei Eingangspins A, B, C_{in} . Die Punkte zeigen Abzweigungen desselben Signals. Kreuzende Leitungen ohne Punkt sind nicht verbunden. X ist kein vierter Eingang: Der Ausgang des ersten XOR geht an das zweite XOR *und* an das rechte AND. A und B gehen jeweils an das erste XOR und an das linke AND; C_{in} geht an das zweite XOR und an das rechte AND.

Warum stimmt der Carry? Entweder sind A und B beide 1, oder genau eines der beiden Bits ist 1 und C_{in} ist ebenfalls 1. Gleichwertig ist $C_{out} = AB + AC_{in} + BC_{in}$: der Majority3-Ausgang.

03 / VIER STUFEN

Ripple-Carry: Der Übertrag wandert

Baue zuerst einen Volladdierer als eigene Schaltung `volladdierer.dig`. Seine Anschlüsse heißen A, B, Cin, S, Cout; alle sind 1 Bit breit. Verwende dann vier Instanzen in `ripple4.dig`.



Hier läuft die Kette zum leichteren Verfolgen von Bit 0 links zu Bit 3 rechts. Beim Schreiben einer Binärzahl bleibt die Reihenfolge wie gewohnt $S_3 S_2 S_1 S_0$.

Eingänge der Gesamtschaltung: A3 A2 A1 A0 B3 B2 B1 B0 Cin. **Ausgänge:** S3 S2 S1 S0 Cout. Separate 1-Bit-Pins vermeiden zunächst zusätzliche Bus- und Splitter-Syntax. In jeder Stufe werden nur die gleich nummerierten A- und B-Bits addiert.

Ein sensibler Carry-Pfad: Setze $A = 1111$, $B = 0000$. Alle Stufen propagieren. Ändere nach dem Einschwingen C_{in} von 0 auf 1. Die Änderung läuft nacheinander über c_1, c_2, c_3 zu c_4 .

Pfad zählen ist genauer als Bausteine zählen. In unserem Volladdierer führt der Weg von C_{in} zu C_{out} über AND und OR, also zwei Gatter. Durch vier Stufen sind es acht Gatter, wenn X jeweils schon stabil ist. Zu S_3 sind es drei Carry-Stufen plus ein XOR: sieben Gatter. Wenn A oder B wechselt, kann am Anfang zusätzlich ein XOR auf dem Pfad liegen.

Für diese konkrete 5-Gatter-Struktur und gleiche Gatterlaufzeit t_g kann ein A/B-Pfad zu C_{out} neun Gatter durchlaufen. Beispiel: $B_0 = 0$, höhere Stufen propagieren, $C_{in} = 1$, A_0 wechselt. **Immer Startsignal, Zielsignal und Annahmen nennen.**

Vereinfachtes Blockmodell im Python-Teil:

$$t_{ripple}(N) = N t_{FA}.$$

t_{FA} ist eine angenommene effektive Stufenzeit. Die Formel erklärt lineares Wachstum und ersetzt keine genaue Timing-Analyse aller Gatterpfade.

04 / VORAUSBERECHNEN

Generate, Propagate und Lookahead

Für das Lesen des Buches verwenden wir dessen OR-Definition:

$$G_i = A_i B_i, \quad P_i = A_i + B_i, \quad c_{i+1} = G_i + P_i c_i.$$

A_i	B_i	G_i	P_i	Wirkung
0	0	0	0	Carry wird gestoppt
0	1	0	1	Carry wird weitergegeben
1	0	0	1	Carry wird weitergegeben
1	1	1	1	Carry wird unabhängig von c_i erzeugt

Achtung auf zwei Definitionen: Manche Darstellungen verwenden $P_i = A_i \oplus B_i$. Auch das funktioniert in der Carry-Gleichung. Hier ist P_i aber OR, und daher gilt *nicht* $S_i = P_i \oplus c_i$. Für die Summe bleibt $S_i = A_i \oplus B_i \oplus c_i$.

Einsetzen macht den nächsten Übertrag direkt von früheren Eingängen abhängig:

$$\begin{aligned} c_1 &= G_0 + P_0 c_0, \\ c_2 &= G_1 + P_1 c_1 \\ &= G_1 + P_1 (G_0 + P_0 c_0) \\ &= G_1 + P_1 G_0 + P_1 P_0 c_0. \end{aligned}$$

Die Hardware kann diese Terme parallel vorbereiten. Sie muss dafür zusätzliche Gatter und Verbindungen bereitstellen.

Nicht jeder CLA wächst logarithmisch. Der einfache Block-CLA im Buch berechnet Carry innerhalb eines Blocks schneller, reicht es zwischen den Blöcken aber weiterhin nacheinander weiter. Bei fester Blockgröße bleibt das Wachstum mit N linear, nur mit günstigeren Konstanten (Buch S. 239–241).

Hierarchisches Lookahead / Prefix: Gruppen werden stufenweise zusammengeführt. Nach einer Ebene kennt man Gruppen von 2 Bits, dann 4, dann 8 usw. Für 32 Bits sind fünf solcher Ebenen nötig. Bei nicht-potenzförmigen Bitbreiten runden wir die Ebenenzahl auf:

$$L(N) = \lceil \log_2 N \rceil.$$

Mehr Parallelität bedeutet zusätzliche Logik und Verdrahtung. Reale Geschwindigkeit hängt auch von Lasten, Leitungslängen und Gattergrößen ab. Ein größeres Netz ist nicht in jeder kleinen Anwendung die bessere Wahl.

05 / MODELL STATT MESSUNG

Zwei Kurven, zwei kleine Funktionen

Der Jahresplan nennt die zweite Funktion `delay_cla`. In unseren Dateien steht dieser Name für ein **idealisiertes hierarchisches Lookahead-/Prefix-Modell**, nicht für den linear verketteten Block-CLA.

$$t_{ripple} = N t_{FA}, \quad t_{cla} = (\lceil \log_2 N \rceil + 2) t_{Stufe}.$$

Die zwei zusätzlichen Stufen stehen pauschal für Vorbereitung und Summenbildung. Alle Prefix-Ebenen erhalten im Lehrmodell dieselbe Zeit. Das ist eine Wachstumsillustration, keine gemessene Chipgeschwindigkeit.

Beispielannahmen: $t_{FA} = 2$ ns und $t_{Stufe} = 1$ ns.

N	$\lceil \log_2 N \rceil$	Ripple (ns)	Prefix-Modell (ns)
4	2	8	4
8	3	16	5
16	4	32	6
32	5	64	7
64	6	128	8

Verdoppelt sich N , verdoppelt sich die Ripple-Zeit. Im Prefix-Modell kommt bei einer Verdopplung genau eine Ebene hinzu. Das meint $O(N)$ gegenüber $O(\log N)$; die Schreibweise sagt nichts über absolute Nanosekunden ohne weitere Annahmen.

Dein eigener Python-Anteil: Öffne `praxis/delay_lernen.py`. Ersetze in genau zwei Funktionen die Platzhalter durch eine Rechnung mit `return`. `ceil` und `log2` werden bereits importiert.

```
from math import ceil, log2

# Beispiel fuer die beiden Bausteine:
ebenen = ceil(log2(12)) # 4, denn 8 < 12 <= 16
```

`delay_ripple(4, 2.0)` bedeutet: vier Bits, zwei Nanosekunden angenommene Zeit pro Volladdiererstufe. Die Funktion soll 8.0 zurückgeben. `return` übergibt diesen Wert an den Aufrufer; `print` zeigt ihn nur an.

Ablauf: Funktionen ergänzen, `python3 pruefe_delay.py` starten, dann `python3 plot_delay.py`. Der Plotcode und die Prüfungen sind Hilfsmittel. Du musst heute weder `matplotlib` noch eine neue Schleifensyntax selbst implementieren.

06 / PRÜFEN UND ABSCHLIESSEN

Woran du erkennst, dass es funktioniert

Erst eine Stufe prüfen, dann vier verbinden. Prüfe den Volladdierer für alle acht Kombinationen. Für vier Bits gibt es $16 \cdot 16 \cdot 2 = 512$ Eingabekombinationen inklusive C_{in} .

Der bereitgestellte Python-Prüfer erzeugt Digital-Testdaten aus normaler Integer-Addition. Er prüft danach deine gespeicherte Schaltung im Simulator. Er baut und löst deine Schaltung nicht für dich.

```
cd praxis
python3 pruefe_schaltungen.py schaltungen/volladdiererer.dig
python3 pruefe_schaltungen.py schaltungen/ripple4.dig
```

Die Pin-Namen müssen exakt wie auf Seite 3 heißen. Beide Dateien müssen im selben Schaltungsordner liegen, wenn ripple4 den Volladdierer als Unterbaustein verwendet. Der Prüfer benötigt Java und Digital. Ein abweichender Pfad zu Digital.jar lässt sich über --jar angeben.

Vier schnelle Handkontrollen:

A	B	C_{in}	S	C_{out}
0	0	0	0	0
7	1	0	8	0
15	0	1	0	1
15	15	1	15	1

Ein bestandener Wahrheitstabellentest zeigt die korrekte stationäre Funktion. Er beweist keine bestimmte Laufzeit. Dafür brauchst du einen separat beschriebenen Signalwechsel und ein Zeitdiagramm oder eine Pfadzählung mit klaren Annahmen.

Abschlussnachweise im Praxisordner:

- zwei eigene Schaltungen und erfolgreiche Prüfprotokolle;
- eine markierte Carry-Kette mit Start, Ziel und gezählten Stufen;
- eigener Python-Plot über alle ganzzahligen N von 4 bis 64;
- kurze englische auswertung.md: Was hast du selbst gemacht, was wurde geprüft, was bleibt offen?

Arbeitsbudget: Der Kalender plant 60 Minuten Theorie und 60 Minuten Praxis. Nutze das als Orientierung. Wenn Aufbau und Fehlersuche länger dauern, notiere den Stand. Prefix-Verdrahtung und ein eigener Plotter sind keine Zusatzpflicht.

Lesepfad im Buch: Halb-/Volladdierer und Ripple: S. 238–239; Block-CLA: S. 239–241; Prefix und Abwägung: S. 241–243. PDF-Seite = Buchseite + 23 in diesem Abschnitt. Diese Seiten wurden am lokalen Buch geprüft.